

SmartDelivery Infra — Roadmap

Current cluster: k3s two-node (sd-master + sd-worker-0), Hetzner CX22 × 2, Flannel VXLAN on private 10.0.0.0/16 network.

Phase 1 — Scale Out (prerequisite for load testing)

Goal: Add a second Hetzner node before any load testing. Current single-node utilisation (~75%) leaves no headroom for HPA scale-out under load.

- ☒ Provision second Hetzner node (CX22 or CX32)
 - ☒ Join as k3s worker node (`k3s agent`)
 - ☒ Verify pod scheduling across both nodes (`kubectl get pods -o wide`)
 - ☒ Update `docs/INFRA_SPEC.md` with new node details
 - ☒ Update firewall scripts in `base/` for new node IP
-

Phase 2 — Load Testing (k6)

Goal: Run realistic load against the live cluster, observe HPA scaling, capture Grafana metrics.

- ☒ Review and finalise `release/k6-load-test.js` scenarios
 - ☒ Run k6 against all 5 service endpoints through the Istio IngressGateway
 - ☒ Capture Grafana dashboard during ramp-up (HPA scaling in action)
 - ☒ Document p95 latency and error rate baselines per service (p95 2.41s, 555 iterations)
 - ☒ Screenshot Grafana + HPA output for README / post
-

Phase 3 — Chaos Testing

Goal: Validate mesh resilience using Istio-native fault injection (zero additional tooling).

- ☒ Inject HTTP delay (5s) into `payment-service` — observe Jaeger trace impact
 - ☒ Inject HTTP abort (503) into `order-service` — observe error rate in Grafana
 - ☒ Document recovery times and mesh behaviour (see `docs/CHAOS_TESTING.md`)
 - ☐ ~~Test pod deletion (`kubectl delete pod`)~~ — skipped; ReplicaSet self-healing is Kubernetes core behaviour, not app-specific
-

Phase 4 — Alertmanager Wiring

Goal: Complete the observability loop — alerts fire to a real notification channel.

- ☐ Configure Alertmanager with a Slack or webhook notification target
 - ☐ Verify existing `release/monitoring/alerting_rules.yml` rules fire correctly under load
 - ☐ Test alert → notification end-to-end
 - ☐ Update `docs/OBSERVABILITY_ARCH.md`
-

Phase 5 — Canary Deployments

Goal: Use Istio traffic splitting to roll out a v2 of one service safely.

- ☐ Create v2 deployment for one service (e.g. `restaurent-service`)
- ☐ Add Istio `DestinationRule` with v1 / v2 subsets
- ☐ Add `VirtualService` weight-based routing (90/10 split)
- ☐ Graduate to 100% v2, then remove v1
- ☐ Document the pattern for reuse across all services

Phase 6 — Helm Migration

Goal: Replace raw `kubectl apply` manifests in `release/` with a proper Helm chart.

- ☐ Create `charts/smartdelivery/` Helm chart structure
- ☐ Templatisé all 5 service deployments (shared `deployment.yaml` template)
- ☐ Move configmap values to `values.yaml` (one file to configure all services)
- ☐ Update GHA pipeline to use `helm upgrade --install`
- ☐ Test rollback with `helm rollback`

Phase 7 — GitOps (Flux or ArgoCD)

Goal: Replace `kubectl apply` push model in GHA with a GitOps pull model.

- ☐ Evaluate Flux vs ArgoCD for single-node resource constraints
- ☐ Install chosen tool, point at this repo
- ☐ Remove manual `kubectl apply` steps from `base/gha.yaml`
- ☐ Add sync status badge to README

Backlog (unordered)

- **Persistent Jaeger** — swap in-memory Jaeger for Elasticsearch-backed (ES already running)
- **mTLS audit** — verify and document all inter-service traffic is mTLS via Kiali
- **Rate limiting** — Istio `EnvoyFilter` or `RateLimitService` at the gateway
- **DB migrations in CI** — EF Core migration Job before each deploy
- **SLO definitions** — error budget recording rules in Prometheus per service
- **Multi-region** — explore Hetzner Nuremberg + Helsinki nodes for geo-redundancy
- **Cost tracking** — document actual monthly Hetzner spend breakdown per component